

Web y Patrones Semana 8



¿Cómo va la clase hoy?

Se hará un revisión de 3 formas de hacer consultas en JPA Hibernate.

Se programará todo el ciclo de estas 3 maneras.

Profesionalmente sólo se selecciona una forma, según lo que desee el programador, en el caso de esta clase se realizan las 3 formas para mostrar su funcionamiento.

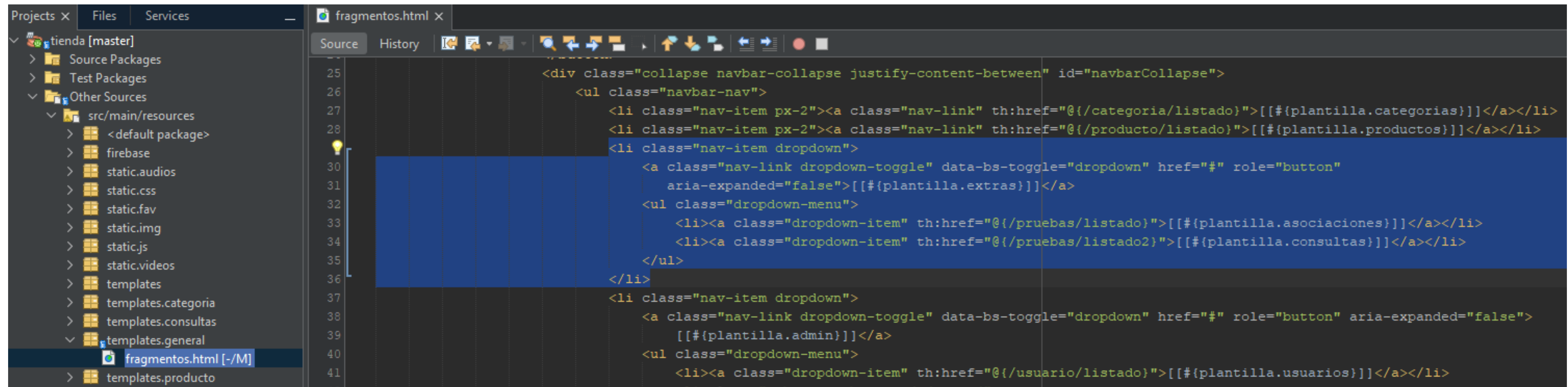
En el proyecto final del curso SOLO una forma se debe utilizar, en producción es innecesario hacer la misma consulta de 3 maneras diferentes.

Haremos:

- Consultas Derivadas
- Consultas JPQL
- Consultas SQL

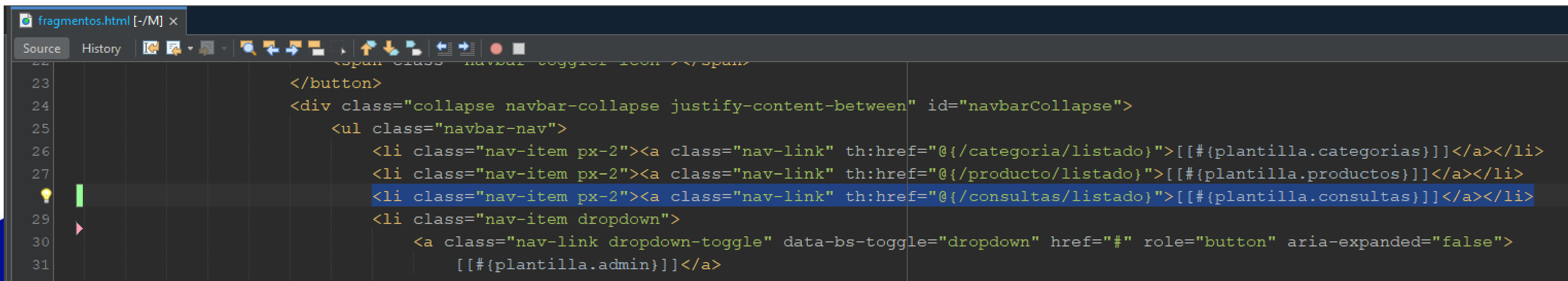
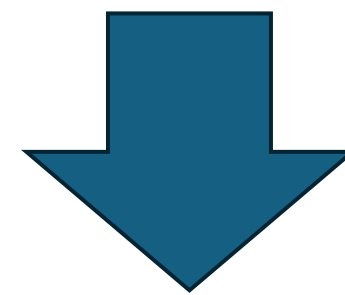


Un ajuste inicial



```
25 <div class="collapse navbar-collapse justify-content-between" id="navbarCollapse">
26   <ul class="navbar-nav">
27     <li class="nav-item px-2"><a class="nav-link" th:href="@{/categoria/listado}">[[#{plantilla.categorias}]]</a></li>
28     <li class="nav-item px-2"><a class="nav-link" th:href="@{/producto/listado}">[[#{plantilla.productos}]]</a></li>
29     <li class="nav-item dropdown">
30       <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button"
31         aria-expanded="false">[[#{plantilla.extras}]]</a>
32       <ul class="dropdown-menu">
33         <li><a class="dropdown-item" th:href="@{/pruebas/listado}">[[#{plantilla.asociaciones}]]</a></li>
34         <li><a class="dropdown-item" th:href="@{/pruebas/listado2}">[[#{plantilla.consultas}]]</a></li>
35       </ul>
36     </li>
37     <li class="nav-item dropdown">
38       <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button" aria-expanded="false">
39         [[#{plantilla.admin}]]</a>
40       <ul class="dropdown-menu">
41         <li><a class="dropdown-item" th:href="@{/usuario/listado}">[[#{plantilla.usuarios}]]</a></li>
```

Cambiamos el menú de extras por una sola consulta



```
23 </button>
24 <div class="collapse navbar-collapse justify-content-between" id="navbarCollapse">
25   <ul class="navbar-nav">
26     <li class="nav-item px-2"><a class="nav-link" th:href="@{/categoria/listado}">[[#{plantilla.categorias}]]</a></li>
27     <li class="nav-item px-2"><a class="nav-link" th:href="@{/producto/listado}">[[#{plantilla.productos}]]</a></li>
28     <li class="nav-item px-2"><a class="nav-link" th:href="@{/consultas/listado}">[[#{plantilla.consultas}]]</a></li>
29     <li class="nav-item dropdown">
30       <a class="nav-link dropdown-toggle" data-bs-toggle="dropdown" href="#" role="button" aria-expanded="false">
31         [[#{plantilla.admin}]]</a>
```

Consultas Derivada, con Filtros y Ordenamiento

- Una necesidad común al desarrollar una aplicación es filtrar datos dentro de un rango específico y presentarlos de forma ordenada.
- Los métodos de consulta derivados (Derived Query Methods) de Spring Data JPA simplifican esta tarea.
- Son métodos, cuyo nombre sigue una convención estricta, permiten al desarrollador realizar consultas complejas sin escribir una sola línea de JPQL o SQL.
- Por ejemplo, en el caso de la tabla producto, es posible buscar todos los productos cuyo precio se encuentre entre un valor mínimo y uno máximo, y al mismo tiempo, ordenar los resultados por precio de manera ascendente.
- Este tipo de consulta, además de ser altamente legible, aprovecha la sintaxis del nombre del método para construir la consulta subyacente de forma automática, haciendo el código más limpio y menos propenso a errores.

`findByPrecioBetweenOrderByPrecioAsc`

Explicación del método

`findByPrecioBetweenOrderByPrecioAsc`

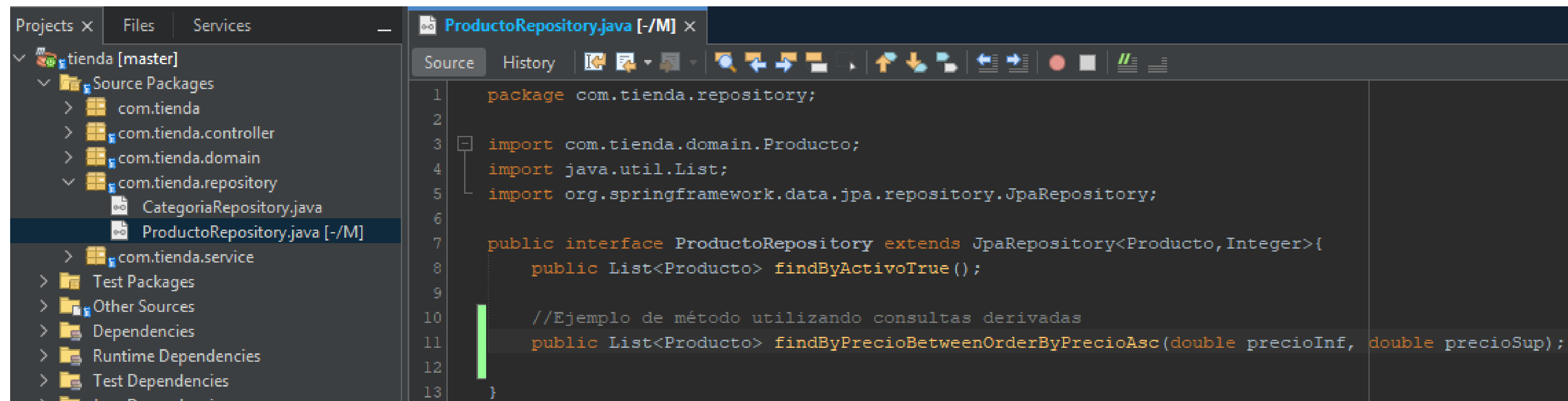
El nombre del método se construye siguiendo una convención estricta que le permite a Spring Data JPA interpretar la intención y generar la consulta requerida. Se puede dividir en las siguientes partes:

1. **findBy**: Es el prefijo estándar que indica que se desea buscar entidades basadas en uno o más criterios.
2. **Precio**: Se refiere al atributo precio de la entidad Producto que se utilizará como criterio de filtro.
3. **Between**: Es el operador de comparación, especifica que la búsqueda debe realizarse dentro de un rango definido por dos parámetros de entrada (un valor inferior y un valor superior).
4. **OrderBy**: Indica que los resultados de la consulta deben ser ordenados.
5. **Precio**: Vuelve a especificar el atributo precio, ahora como el campo por el cual se ordenarán los resultados.
6. **Asc**: Es la dirección del ordenamiento, que en este caso es ascendente (de menor a mayor).

ProductoRepository.java

Este enfoque permite realizar operaciones complejas de filtrado y ordenamiento de manera declarativa, mejorando la legibilidad y mantenibilidad de tu código.

En ProductoService el método se declara de la siguiente manera:



```
1 package com.tienda.repository;
2
3 import com.tienda.domain.Producto;
4 import java.util.List;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 public interface ProductoRepository extends JpaRepository<Producto,Integer>{
8     public List<Producto> findByActivoTrue();
9
10    //Ejemplo de método utilizando consultas derivadas
11    public List<Producto> findByPrecioBetweenOrderByPrecioAsc(double precioInf, double precioSup);
12
13 }
```

Otros ejemplos de consultas derivadas

Introduccion	Ejemplo	JPQL
After	findByFechaVentaAfter	... where x.fechaVenta > ?1
And	findByApellidosAndNombre	... where x.apellidos= ?1 and x.firstname= ?2
Before	findByFechaVentaBefore	... where x.fechaVenta < ?1
Between	findByFechaVentaBetween	... where x.fechaVenta between ?1 and ?2
Containing	findByApellidosContaining	... where x.apellidos like ?1 (el parámetro inicia y termina con %)
EndingWith	findByApellidosEndingWith	... where x.apellidos like ?1 (el parámetro inicia con %)
Equals	findByApellidosEquals	... where x.apellidos= ?1
False	findByActivoFalse()	... where x.activo = false
GreaterThan	findByPrecioGreaterThan	... where x.precio > ?1

Otros ejemplos de consultas derivadas

Introduccion	Ejemplo	JPQL
GreaterThanOrEqualTo	findByPrecioGreaterThanOrEqualTo	... where x.precio >= ?1
IgnoreCase	findByApellidosIgnoreCase	... where UPPER(x.apellidos) = UPPER(?1)
In	findByExistenciasIn(Collection<Existencia>existencias)	... where x.existencias in ?1
Is	findByApellidosIs	... where x.apellidos= ?1
IsNotNull	findByTelefonosIsNotNull	... where x.telefono is not null
IsNull	findByTelefonosIsNull	... where x.telefono is null
LessThan	findByPrecioLessThan	... where x.precio < ?1
LessThanOrEqualTo	findByPrecioLessThanOrEqualTo	... where x.precio <= ?1
Like	findByApellidosLike	... where x.apellidos like ?1

Otros ejemplos de consultas derivadas

Introductor	Ejemplo	JPQL
Not	findByApellidosNotLike	... where x.apellidos <> ?1
NotIn	findByExistenciasNotIn(Collection<Existencia>existencias)	... where x.existencias not in ?1
NotLike	findByApellidosNotLike	... where x.apellidos not like ?1
NotNull	findByTelefonoNotNull	... where x.telefono is not null
Null	findByTelefonoNull	... where x.telefono is null
Or	findByApellidosOrNombre	... where x.apellidos= ?1 or x.firstname= ?2
OrderBy	findByIdCategoriaOrderByDescripcionDesc	... where x.id_categoria = ?1 order by x.descripcion desc
StartingWith	findByApellidosStartingWith	... where x.apellidos like ?1 (el parámetro termina con %)
True	findByActivoTrue()	... where x.activo = true

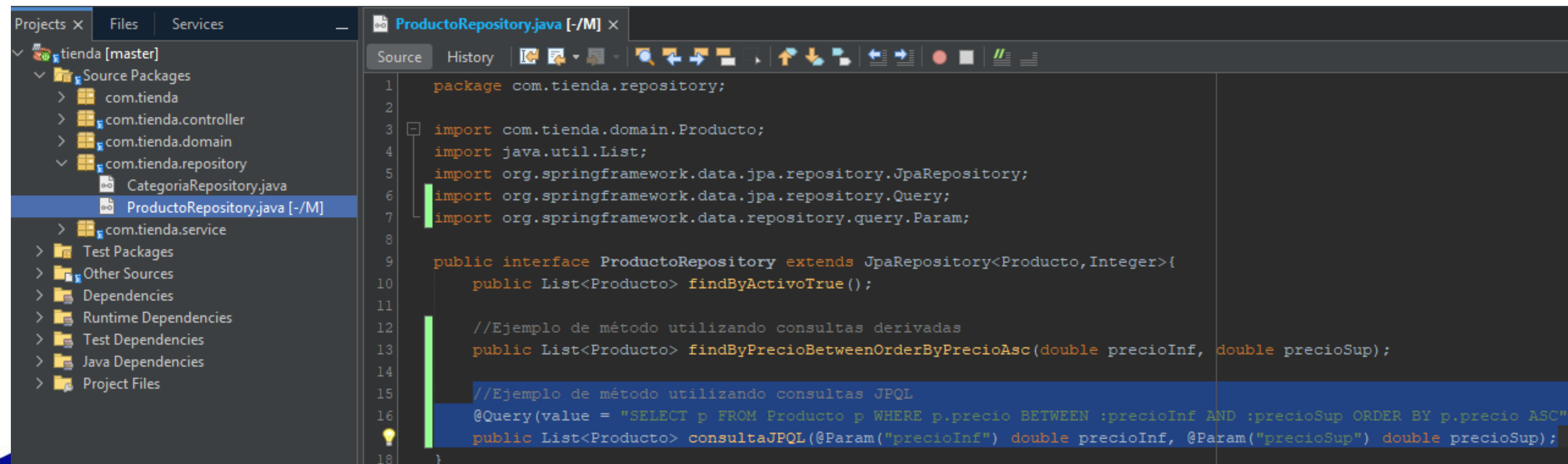
Consultas JPQL, con Filtros y Ordenamiento

- El uso de JPQL (Java Persistence Query Language) para la misma consulta, `findByPrecioBetweenOrderByPrecioAsc`, ofrece un control más explícito sobre la lógica, ya que en lugar de confiar en la convención de nombres, se escribe la consulta directamente. Esto es especialmente útil para consultas más complejas.

ProductoRepository.java

La consulta se define dentro de la anotación **@Query**. Y trabaja directamente sobre las entidades y sus atributos, no sobre las tablas y columnas de la base de datos. **p** es un alias para la entidad **Producto**. La cláusula **WHERE p.precio BETWEEN :precioInf AND :precioSup** filtra los resultados entre los dos parámetros. Finalmente, **ORDER BY p.precio ASC** asegura que los resultados se ordenen por el precio de manera ascendente.

En ProductoService el método se declara de la siguiente manera:



```
1 package com.tienda.repository;
2
3 import com.tienda.domain.Producto;
4 import java.util.List;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 public interface ProductoRepository extends JpaRepository<Producto,Integer>{
10     public List<Producto> findByActivoTrue();
11
12     //Ejemplo de método utilizando consultas derivadas
13     public List<Producto> findByPrecioBetweenOrderByPrecioAsc(double precioInf, double precioSup);
14
15     //Ejemplo de método utilizando consultas JPQL
16     @Query(value = "SELECT p FROM Producto p WHERE p.precio BETWEEN :precioInf AND :precioSup ORDER BY p.precio ASC")
17     public List<Producto> consultaJPQL(@Param("precioInf") double precioInf, @Param("precioSup") double precioSup);
18 }
```

Consultas SQL, con Filtros y Ordenamiento

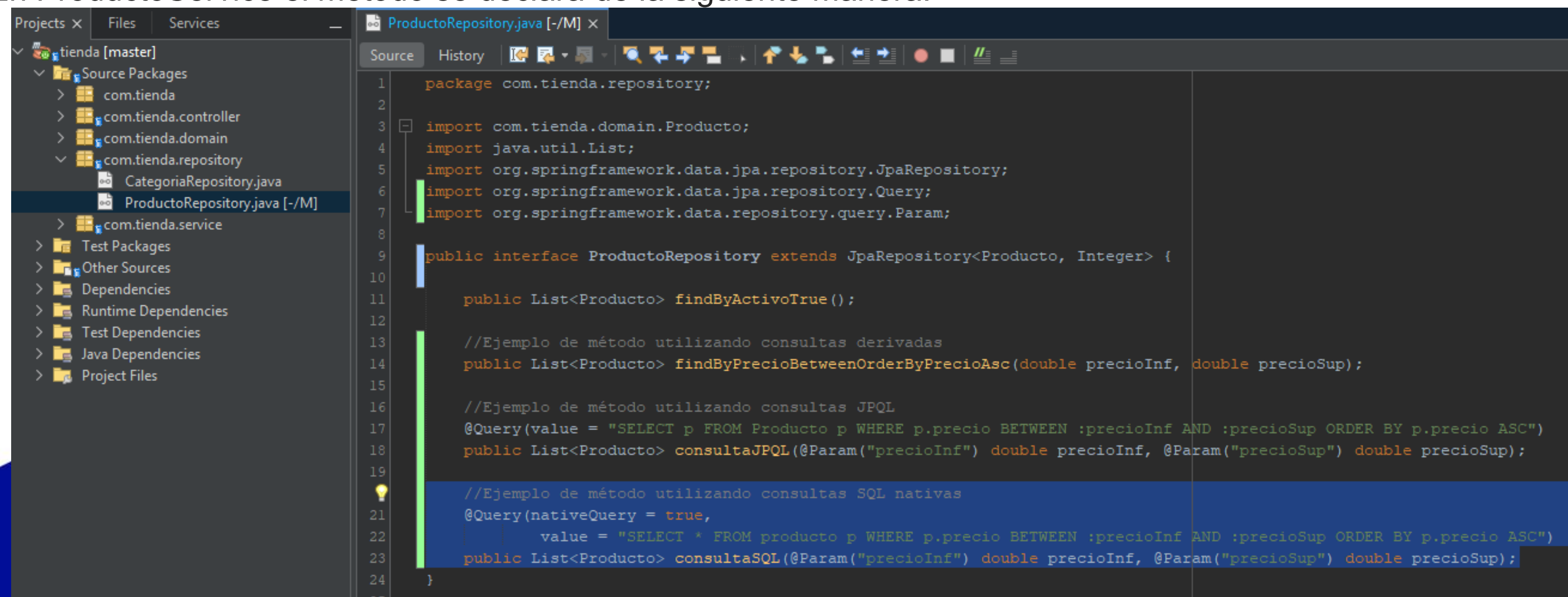
- Usar una consulta SQL nativa es la opción más directa cuando se requiere aprovechar las funcionalidades específicas de la base de datos particular o si se prefiere trabajar directamente con el lenguaje de la base de datos. Esto es distinto a JPQL, que opera sobre las entidades de Java, ya que la consulta nativa se ejecuta directamente en la base de datos

ProductoRepository.java

La consulta SQL nativa que logra el mismo resultado de buscar productos por rango de precio y ordenarlos de forma ascendente es una traducción directa del lenguaje de la base de datos que se ejecutaría en tu motor de base de datos (por ejemplo, MySQL, PostgreSQL, etc.).

Para usar esta consulta en el repositorio JPA, simplemente se agrega la anotación `@Query` al método y se establece el atributo `nativeQuery=true`. Los parámetros se mapean automáticamente por su nombre.

En ProductoService el método se declara de la siguiente manera:



```
1 package com.tienda.repository;
2
3 import com.tienda.domain.Producto;
4 import java.util.List;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 public interface ProductoRepository extends JpaRepository<Producto, Integer> {
10
11     public List<Producto> findByActivoTrue();
12
13     //Ejemplo de método utilizando consultas derivadas
14     public List<Producto> findByPrecioBetweenOrderByPrecioAsc(double precioInf, double precioSup);
15
16     //Ejemplo de método utilizando consultas JPQL
17     @Query(value = "SELECT p FROM Producto p WHERE p.precio BETWEEN :precioInf AND :precioSup ORDER BY p.precio ASC")
18     public List<Producto> consultaJPQL(@Param("precioInf") double precioInf, @Param("precioSup") double precioSup);
19
20     //Ejemplo de método utilizando consultas SQL nativas
21     @Query(nativeQuery = true,
22           value = "SELECT * FROM producto p WHERE p.precio BETWEEN :precioInf AND :precioSup ORDER BY p.precio ASC")
23     public List<Producto> consultaSQL(@Param("precioInf") double precioInf, @Param("precioSup") double precioSup);
24 }
```

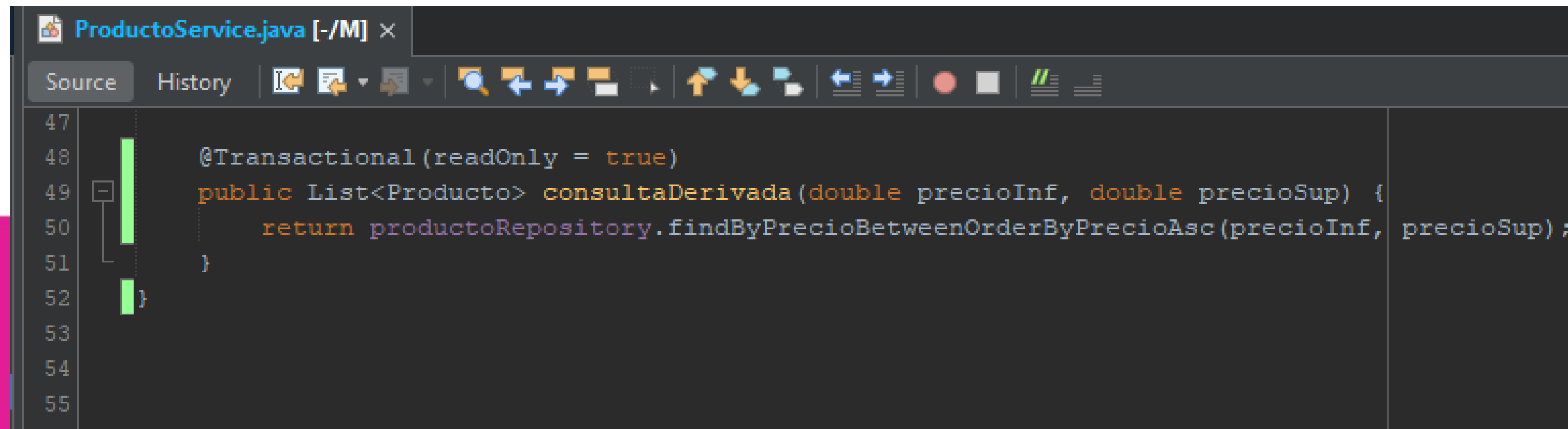
Un ajuste en ProductService

El siguiente método es la manera cómo se implementa el patrón de diseño **Repository** en una aplicación Java. Su función principal es buscar productos en una base de datos que tengan un precio dentro de un rango específico.

El método tiene dos elementos clave: la anotación `@Transactional(readonly=true)` y el uso de un repositorio.

- La anotación **@Transactional** se usa para gestionar transacciones de base de datos. La propiedad **readonly=true** indica que esta transacción solo va a realizar operaciones de lectura. Esto tiene varios beneficios:
 - Rendimiento mejorado: El motor de la base de datos puede aplicar optimizaciones para las consultas de solo lectura, lo que acelera el proceso.
 - Seguridad: Previene la modificación o eliminación de datos, no lo permite.
 - Transparencia: Deja claro que el método no va a alterar el estado de la base de datos.
- Uso del Repositorio `productoRepository`, el método `consultaDerivada` delega la lógica de la consulta a un objeto llamado `productoRepository`. Este objeto es un repositorio, que actúa como un intermediario entre la aplicación y la base de datos.

En resumen, el método `consultaDerivada` utiliza `productoRepository` para ejecutar una consulta de lectura optimizada que encuentra y devuelve una lista de productos cuyos precios se encuentran en el rango especificado, ordenados del más barato al más caro.



```
47
48     @Transactional(readonly = true)
49     public List<Producto> consultaDerivada(double precioInf, double precioSup) {
50         return productoRepository.findByPrecioBetweenOrderByPrecioAsc(precioInf, precioSup);
51     }
52
53
54
55
```

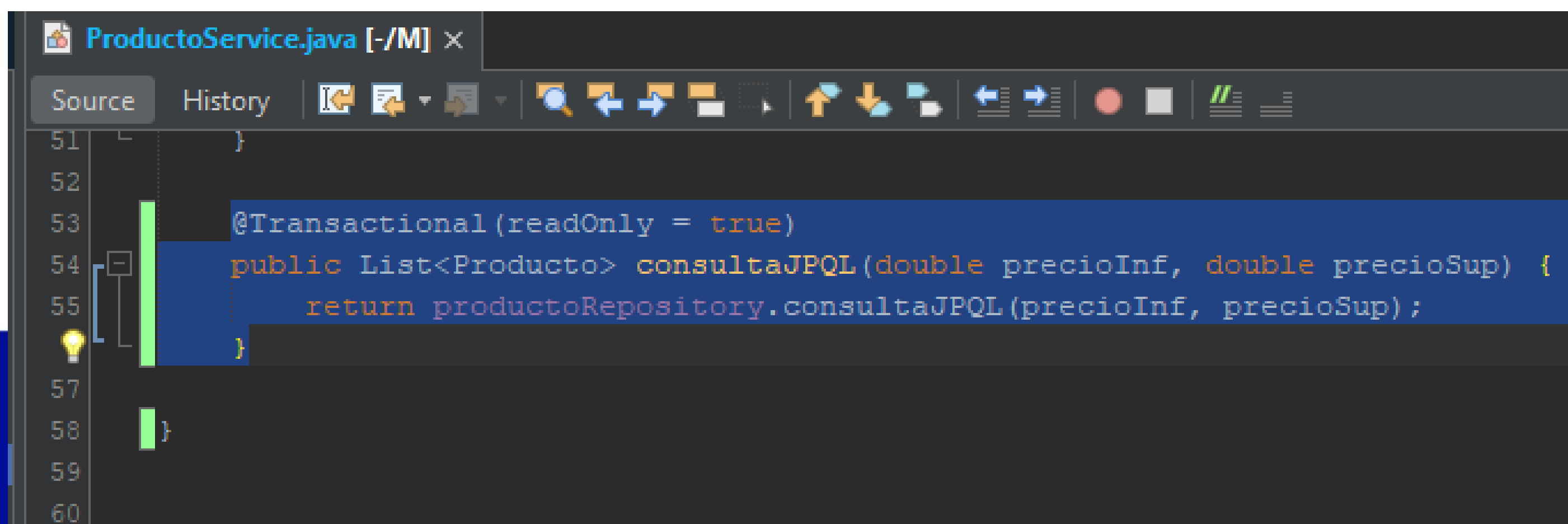

Un ajuste en ProductoService (otro método)

Este método es muy similar al anterior, pero con una diferencia clave en cómo se realiza la consulta a la base de datos. Utiliza una consulta JPQL (Java Persistence Query Language), que ofrece mayor control y flexibilidad que los métodos de consulta derivados.

En lugar de depender de Spring para generar la consulta a partir del nombre del método, **productoRepository.consultaJPQL** invoca un método que ha sido definido explícitamente con una consulta JPQL en la interfaz del repositorio

Este método ejecuta la consulta JPQL predefinida, pasando los parámetros de precio. Esto permite escribir lógica de búsqueda más compleja que no podría expresarse fácilmente con los métodos derivados.

En resumen, mientras que el método consultaDerivada se apoya en una convención de nombres, consultaJPQL delega a una consulta escrita a mano que ofrece mayor control sobre la lógica de la búsqueda. Ambos métodos logran el mismo objetivo de forma segura y optimizada, pero con un enfoque diferente para la construcción de la consulta.



```
ProductoService.java [-/M] x
Source History
51 }
52
53 @Transactional(readonly = true)
54 public List<Producto> consultaJPQL(double precioInf, double precioSup) {
55     return productoRepository.consultaJPQL(precioInf, precioSup);
56 }
57
58 }
59
60
```

Un ajuste en ProductoService (otro método más)

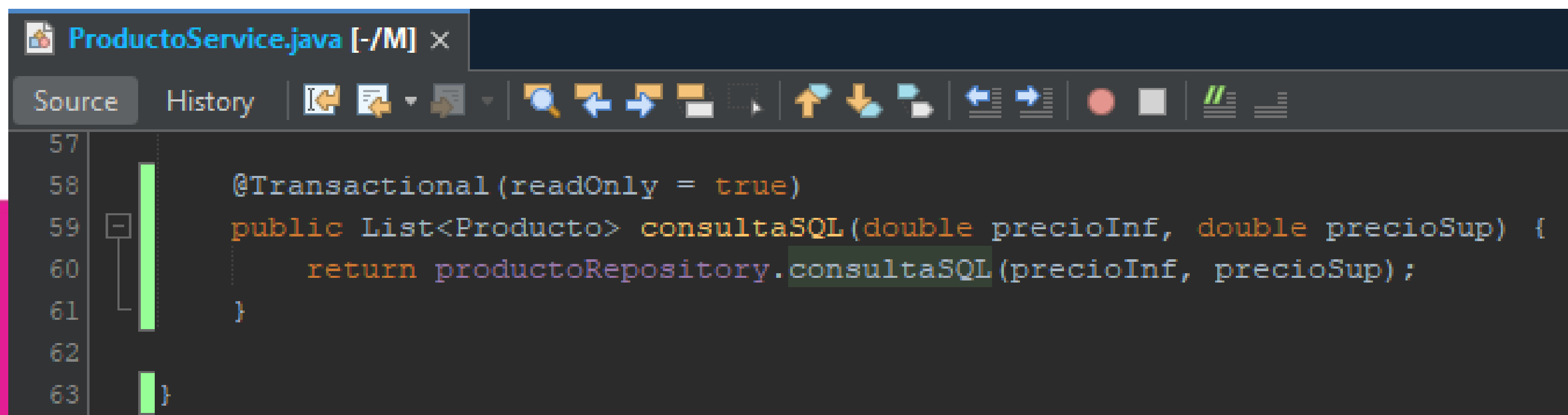
Este método es similar a los anteriores, pero se distingue por el uso de una consulta SQL nativa, que es la forma más directa de interactuar con la base de datos.

El método `consultaSQL` sigue la misma estructura de los ejemplos previos, pero utiliza una consulta escrita en SQL puro en lugar de JPQL o un método derivado. A diferencia de JPQL que trabaja con entidades (objetos Java), el SQL nativo interactúa directamente con las tablas y columnas de la base de datos.

Ventajas: Permite usar funciones específicas del motor de base de datos (por ejemplo, PostgreSQL, MySQL) que no están disponibles en JPQL. Es ideal para consultas complejas, reportes o para optimizar el rendimiento de manera precisa.

Consideraciones: El SQL nativo no es portable entre diferentes tipos de bases de datos y no se beneficia de la abstracción de JPA, lo que requiere un manejo más cuidadoso del mapeo entre los resultados de la consulta y las entidades Java.

En resumen, mientras que los métodos anteriores se basan en abstracciones de JPA, `consultaSQL` es la opción más flexible y de bajo nivel. Ofrece un control total sobre la consulta, a costa de perder la portabilidad entre bases de datos.



```
ProductoService.java [-/M] x
Source History
57
58     @Transactional(readonly = true)
59     public List<Producto> consultaSQL(double precioInf, double precioSup) {
60         return productoRepository.consultaSQL(precioInf, precioSup);
61     }
62
63 }
```


Creando ConsultasController

Ahora que se tiene la lógica de negocio para las tres consultas creadas en el repositorio y en el servicio, el siguiente paso es exponer esta funcionalidad a través de la web. Para ello, se crean los endpoints necesarios en **ConsultaController**, permitiendo que nuestra aplicación reciba peticiones HTTP y utilice los métodos de consulta definidos

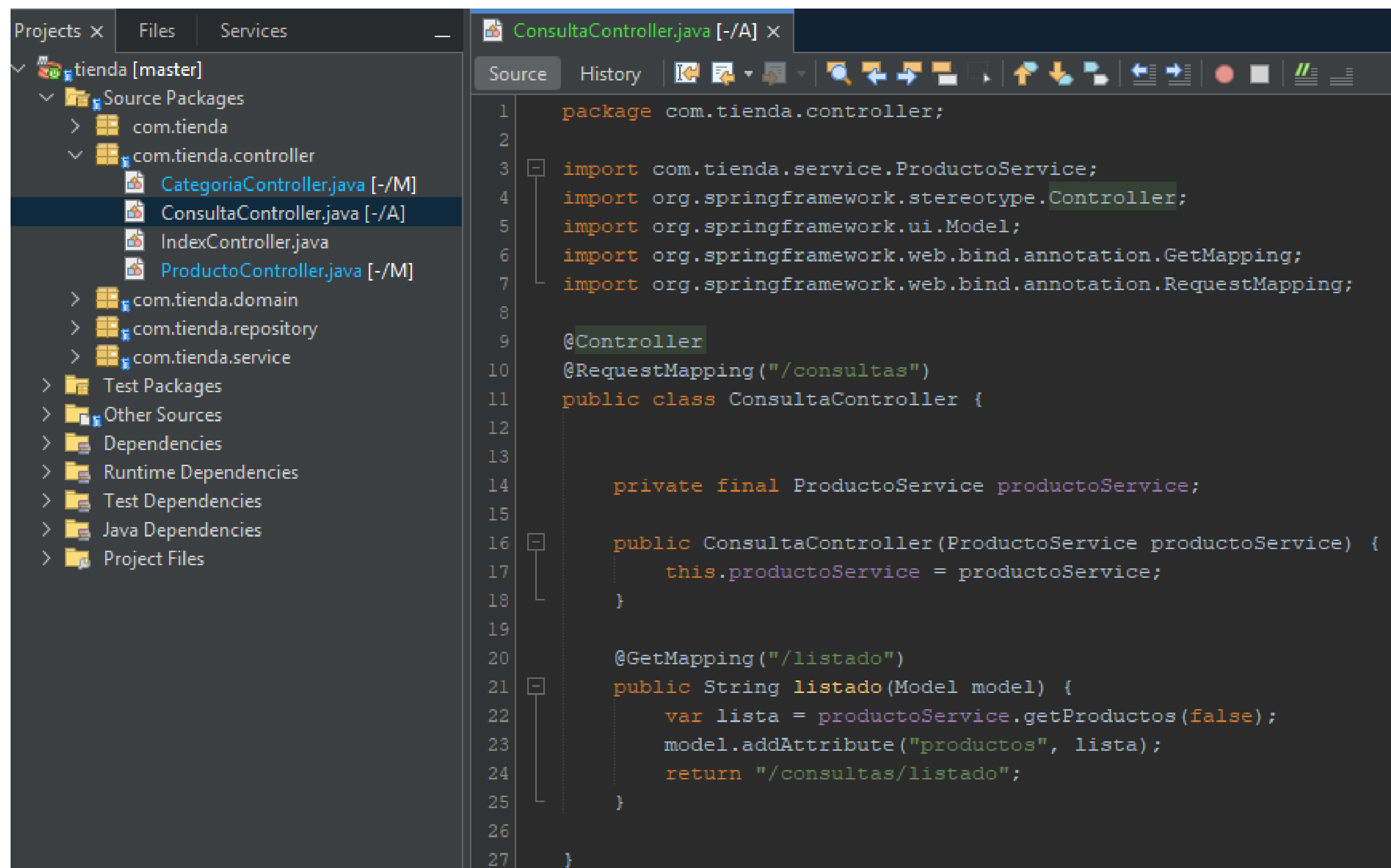


ConsultasController (método listado)

El método **listado** es un controlador web que se encarga de servir una página web con una lista de productos. Funciona como un **endpoint** o punto de acceso para que los clientes (navegadores) puedan solicitar y ver información

El método **listado** es una pieza fundamental del patrón de diseño Modelo-Vista-Controlador (MVC) que utilizamos. Actúa como un controlador web, gestionando la interacción entre el cliente (un navegador) y el servidor para mostrar datos.

Este método no solo muestra datos, sino que encapsula todo el ciclo de vida de una petición web: recibir la solicitud, obtener la información necesaria de la base de datos, y presentarla de manera organizada al usuario. Es un ejemplo claro de cómo se desacopla la lógica de negocio de la interfaz de usuario, haciendo el código más modular y fácil de mantener.



The screenshot shows an IDE with two panels. The left panel displays the project structure for 'tienda [master]'. The right panel shows the source code of 'ConsultasController.java'.

```
package com.tienda.controller;

import com.tienda.service.ProductoService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/consultas")
public class ConsultasController {

    private final ProductoService productoService;

    public ConsultasController(ProductoService productoService) {
        this.productoService = productoService;
    }

    @GetMapping("/listado")
    public String listado(Model model) {
        var lista = productoService.getProductos(false);
        model.addAttribute("productos", lista);
        return "/consultas/listado";
    }
}
```

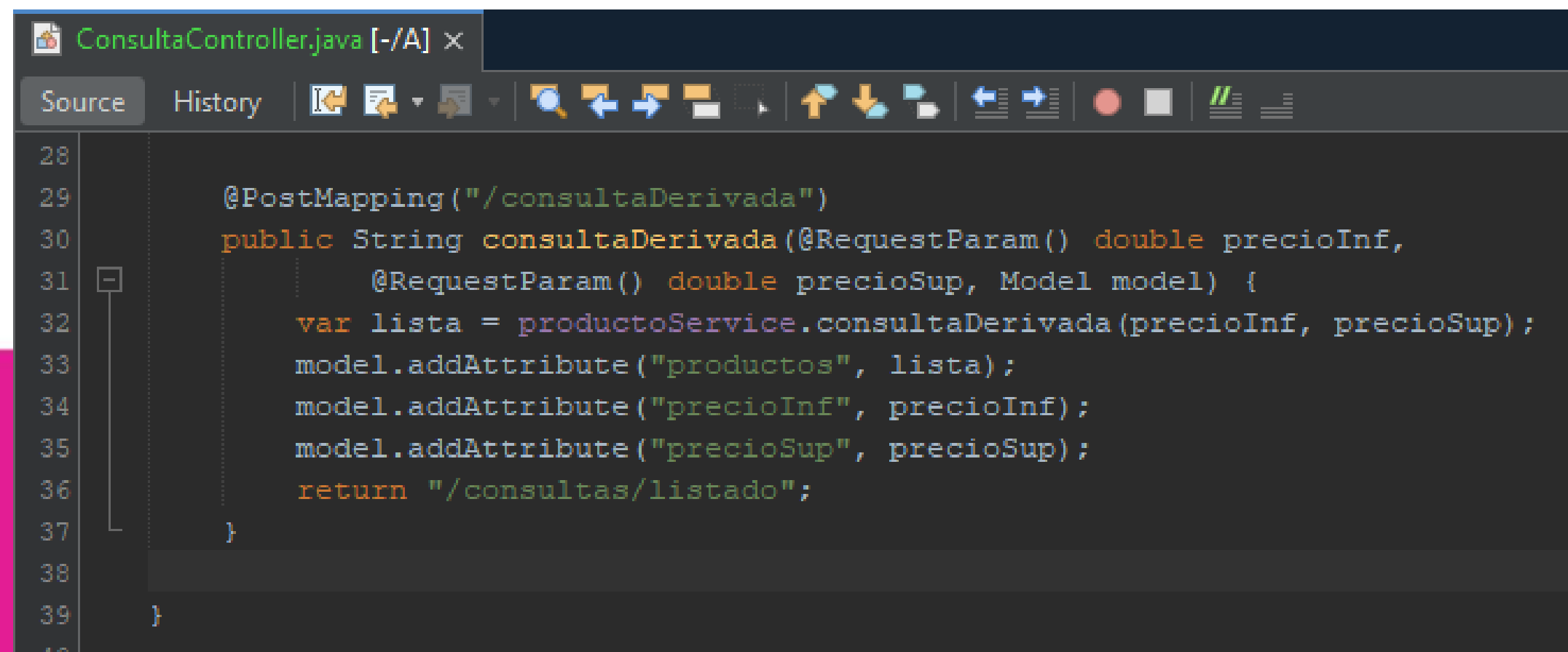
ConsultasController (método consultaDerivada)

El método **consultaDerivada** es un controlador web diseñado para manejar peticiones de filtrado de datos. A diferencia de un simple listado, este método es dinámico y se activa cuando un usuario envía datos desde un formulario, como los precios mínimo y máximo para una búsqueda.

@PostMapping("/consultaDerivada"): Esta anotación mapea las peticiones HTTP de tipo POST a la URL /pruebas/consultaDerivada. Se utiliza POST porque la petición está enviando datos al servidor, lo cual es la práctica estándar al manejar formularios.

return "/consultas/listado";: Se retorna la misma vista que el controlador del listado completo. Esto crea una experiencia de usuario fluida, ya que la página se "refresca" con la lista de productos filtrada, en lugar de redirigir a una página completamente diferente.

El método **consultaDerivada** muestra un flujo de trabajo dinámico: a) recibe datos del usuario, b) los utiliza para ejecutar una consulta específica y c) muestra los resultados en la misma página. Es un ejemplo práctico de cómo se implementan funciones de búsqueda en una aplicación web, utilizando el patrón MVC para separar la lógica de procesamiento (el controlador) de la presentación (la vista).



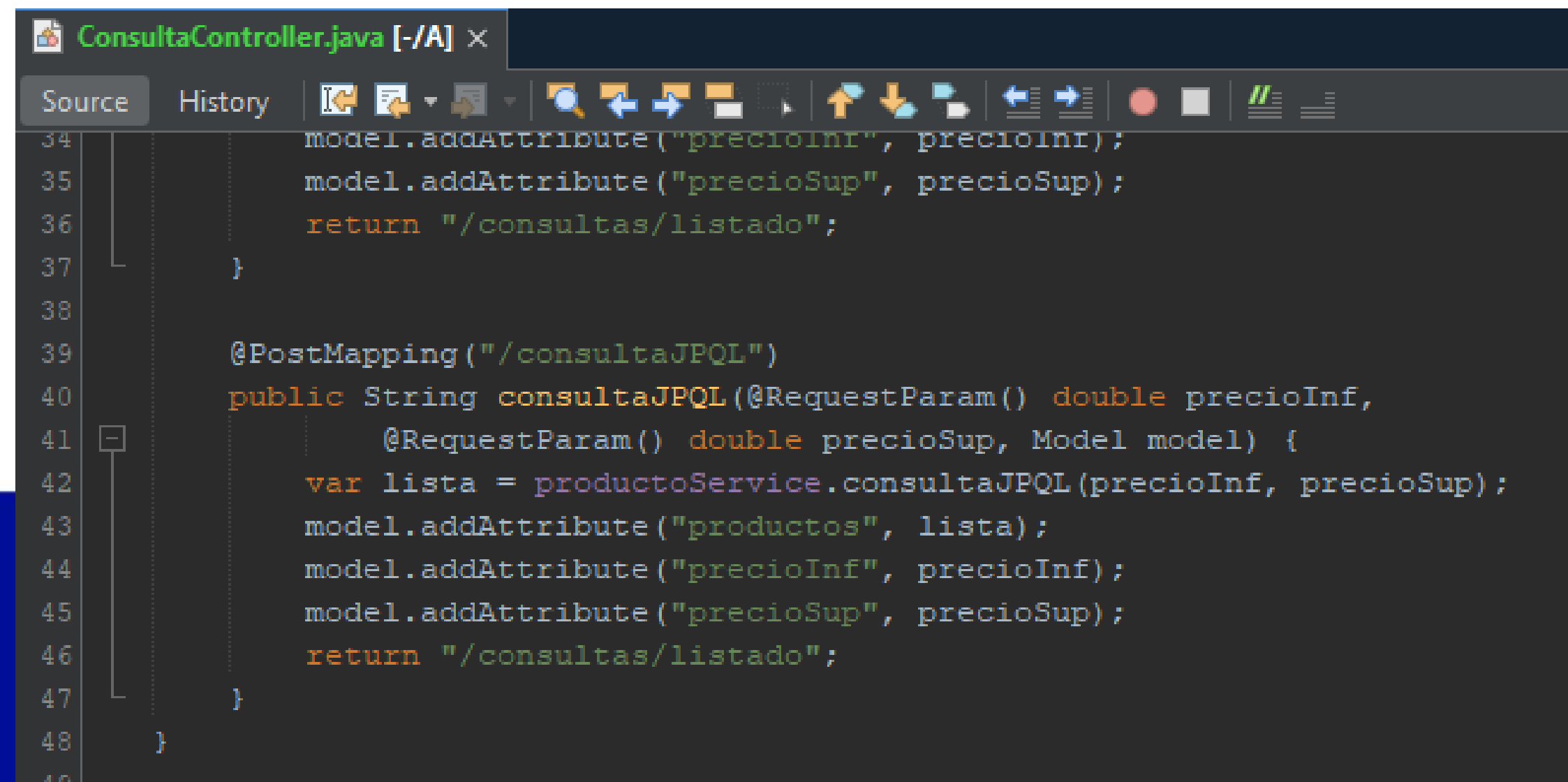
```
ConsultasController.java [-/A] x
Source History
28
29     @PostMapping("/consultaDerivada")
30     public String consultaDerivada(@RequestParam() double precioInf,
31                                   @RequestParam() double precioSup, Model model) {
32         var lista = productoService.consultaDerivada(precioInf, precioSup);
33         model.addAttribute("productos", lista);
34         model.addAttribute("precioInf", precioInf);
35         model.addAttribute("precioSup", precioSup);
36         return "/consultas/listado";
37     }
38
39 }
```


ConsultasController (método consultaJPQL)

El método `consultaJPQL` es un controlador web que, al igual que el método `consultaJPQL` analizado antes, procesa los datos de un formulario para realizar una búsqueda filtrada. Su propósito es manejar una petición HTTP POST y usar los parámetros de la solicitud para ejecutar una consulta JPQL en la capa de servicio.

`var lista = productoService.consultaJPQL(precioInf, precioSup);`: El controlador delega la lógica de negocio a la capa de servicio, llamando al método `consultaJPQL`. Este método se encarga de ejecutar la consulta JPQL predefinida en el repositorio con los parámetros proporcionados, obteniendo la lista de productos que cumplen con el criterio de precio.

El método `consultaJPQL` ejemplifica el patrón MVC en acción, demostrando cómo el controlador maneja una petición con datos, delega la lógica de consulta al servicio y prepara la vista con los resultados. La única diferencia funcional con el método `consultaDerivada` es la consulta que se ejecuta en la capa de servicio: en lugar de usar un método de consulta derivado, se invoca una consulta JPQL predefinida. Esto subraya la flexibilidad de Spring para manejar diferentes tipos de consultas a la base de datos a través de la misma estructura de controlador.



```
ConsultaController.java [-/A] x
Source History
34 model.addAttribute("precioInf", precioInf);
35 model.addAttribute("precioSup", precioSup);
36 return "/consultas/listado";
37 }
38
39 @PostMapping("/consultaJPQL")
40 public String consultaJPQL(@RequestParam() double precioInf,
41 @RequestParam() double precioSup, Model model) {
42     var lista = productoService.consultaJPQL(precioInf, precioSup);
43     model.addAttribute("productos", lista);
44     model.addAttribute("precioInf", precioInf);
45     model.addAttribute("precioSup", precioSup);
46     return "/consultas/listado";
47 }
48 }
```

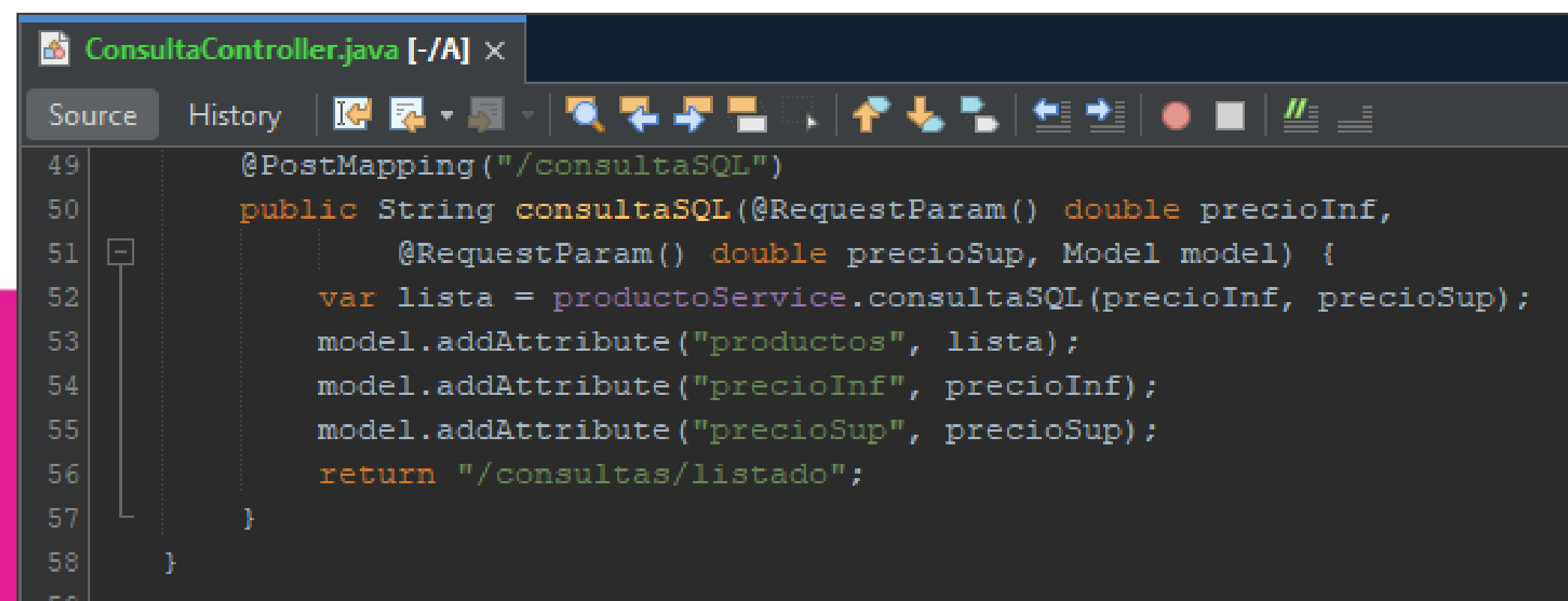
ConsultasController (método consultaSQL)

El método `consultaSQL` es un controlador web que, al igual que los métodos anteriores, gestiona una petición de búsqueda de productos. Su particularidad radica en que delega la tarea a un servicio que ejecuta una consulta SQL nativa, la cual interactúa directamente con la base de datos. Este método demuestra cómo un controlador puede recibir datos de un formulario y utilizarlos para invocar una consulta SQL nativa.

`public String consultaSQL(...)`: La firma del método define que acepta un objeto `Model` y dos parámetros, `precioInf` y `precioSup`, extraídos de la petición HTTP gracias a la anotación `@RequestParam`.

`model.addAttribute(...)`: Aquí, el controlador añade la lista de productos filtrada ("productos") y los valores de los parámetros de búsqueda ("precioInf" y "precioSup") al objeto `Model`. Esto asegura que la vista tenga toda la información necesaria para mostrar los resultados de la búsqueda y los valores usados en el formulario.

El método `consultaSQL` sigue el mismo patrón MVC que los anteriores, recibiendo datos del usuario y pasándolos al servicio para su procesamiento. La diferencia radica en el tipo de consulta que se realiza: una consulta **SQL nativa** en lugar de una consulta derivada o JPQL. Esto permite un control más preciso sobre la base de datos y la ejecución de sentencias específicas del motor de la base de datos, aunque a costa de una menor portabilidad.



```
ConsultasController.java [-/A] x
Source History
49 @PostMapping("/consultaSQL")
50 public String consultaSQL(@RequestParam() double precioInf,
51 @RequestParam() double precioSup, Model model) {
52     var lista = productoService.consultaSQL(precioInf, precioSup);
53     model.addAttribute("productos", lista);
54     model.addAttribute("precioInf", precioInf);
55     model.addAttribute("precioSup", precioSup);
56     return "/consultas/listado";
57 }
58 }
```


Las vistas

Definida la capa de control, que es la que se encarga de recibir las peticiones, procesarlas y preparar los datos. El siguiente paso es pasar a la capa de vista, MVC que tiene la responsabilidad de presentar los datos al usuario.

El enfoque es la creación de las plantillas HTML que utilizan Thymeleaf para:

- **Mostrar los resultados** de las consultas en una tabla.
- **Permitir la interacción** del usuario a través de los formularios.

El objetivo es conectar los datos que preparan los controladores con la interfaz visual que ve el usuario, cerrando así el ciclo completo de nuestra aplicación web.



listado.html

Este código HTML es una plantilla maestra o un layout que define la estructura principal de una página web. Su propósito es ensamblar una vista completa a partir de fragmentos reutilizables, lo que hace que el desarrollo sea más modular y eficiente.

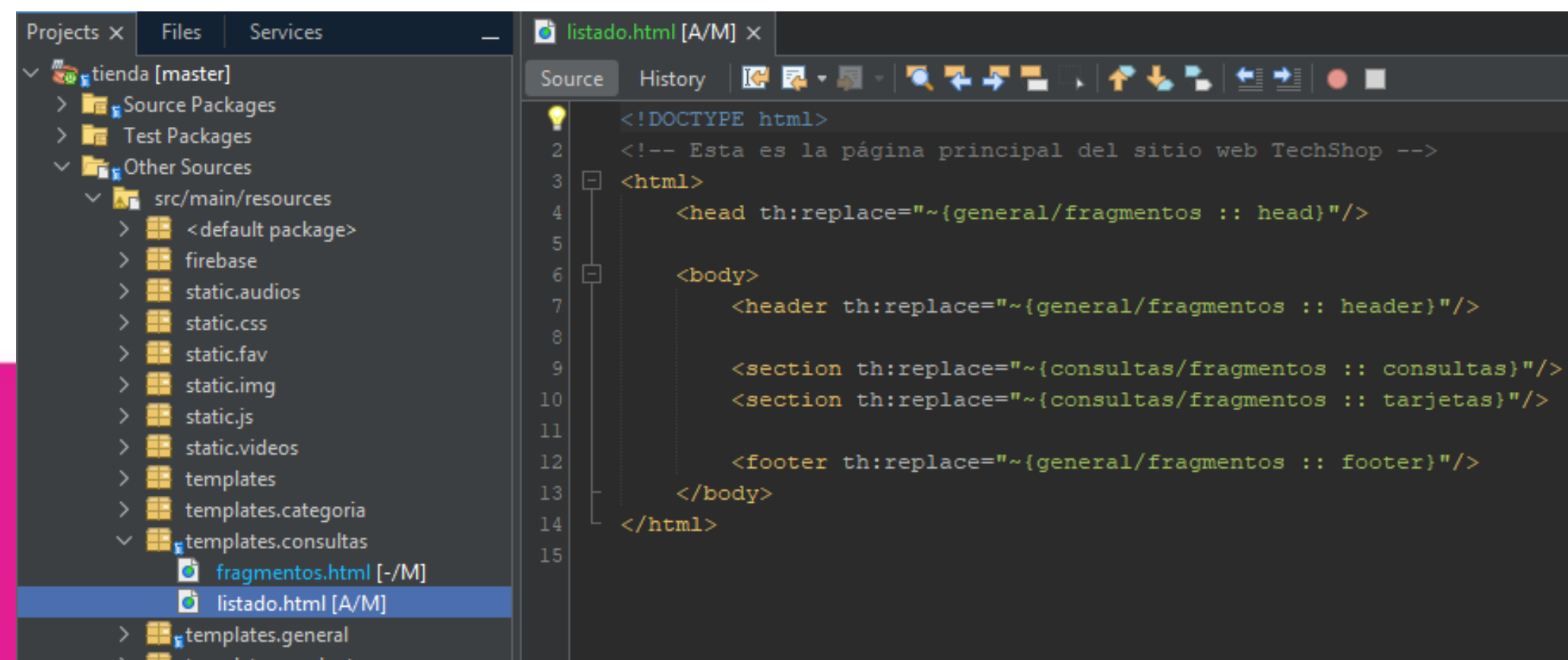
Actúa como un esqueleto que define el orden y la posición de los componentes de la página. Utiliza el motor de plantillas Thymeleaf para incluir partes de código que están en otros archivos. Esto nos permite:

- **Reutilizar** componentes como el encabezado y el pie de página en múltiples vistas.
- **Mantener** el código de forma más sencilla, ya que un cambio en un fragmento se refleja en todas las páginas que lo usan.

`<section th:replace="... :: consultas"/>`: Aquí se carga el fragmento que contiene los formularios de búsqueda. Esto permite al usuario interactuar con la aplicación para realizar consultas.

`<section th:replace="... :: tarjetas"/>`: Inserta el fragmento que muestra los resultados de la búsqueda en un formato visual atractivo (como tarjetas).

En resumen, esta plantilla es un claro ejemplo de diseño modular. En lugar de ser una página estática, es un punto de ensamblaje que combina diferentes componentes, permitiendo un desarrollo más limpio y organizado.

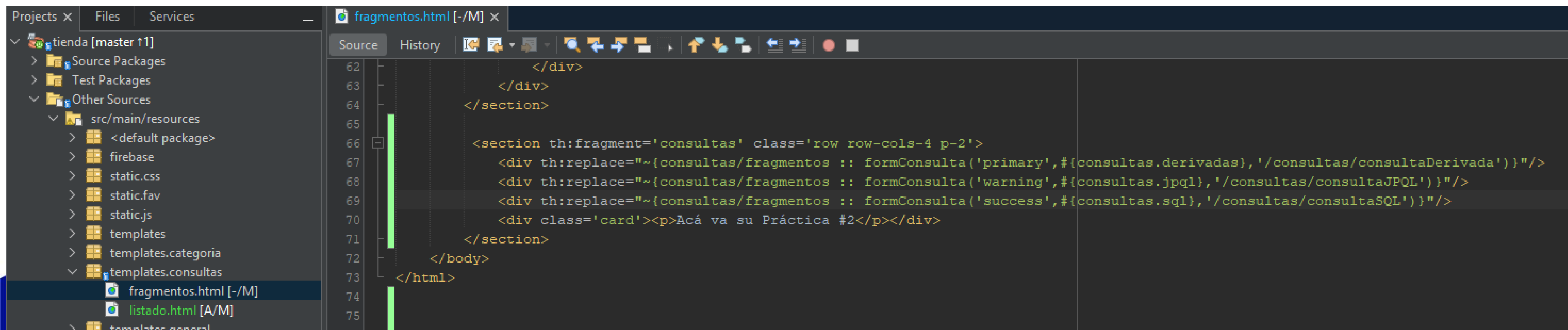


Fragmento consultas (en consultas\fragmentos)

Este fragmento HTML, potencia Thymeleaf, actúa como un contenedor que organiza y muestra una serie de formularios de consulta en una página web. Su principal objetivo es reutilizar otro fragmento de código, **formularioConsulta**, para crear múltiples formularios de búsqueda con diferentes configuraciones.

La sección principal de este fragmento son las cuatro líneas que usan **th:replace**. Cada una inserta el contenido de otro fragmento, **formConsulta**, con diferentes parámetros.

Este fragmento es un excelente ejemplo de la filosofía "*Don't Repeat Yourself*" (DRY). En lugar de escribir tres formularios idénticos con pequeñas variaciones, se utiliza un único fragmento parametrizado (**formConsulta**) y se lo invoca múltiples veces con los valores necesarios para cada caso. Esto hace que el código sea más limpio, fácil de leer y mucho más simple de mantener. Si se necesita hacer un cambio en la estructura de los formularios (por ejemplo, añadir un nuevo campo), solo se tendría que modificar el fragmento **formConsulta**, y el cambio se aplicaría automáticamente a todos los formularios en la página.



```
62         </div>
63     </div>
64 </section>
65
66     <section th:fragment='consultas' class='row row-cols-4 p-2'>
67         <div th:replace="~{consultas/fragmentos :: formConsulta('primary',#{consultas.derivadas},'/consultas/consultaDerivada')}"/>
68         <div th:replace="~{consultas/fragmentos :: formConsulta('warning',#{consultas.jpql},'/consultas/consultaJPQL')}"/>
69         <div th:replace="~{consultas/fragmentos :: formConsulta('success',#{consultas.sql},'/consultas/consultaSQL')}"/>
70         <div class='card'><p>Acá va su Práctica #2</p></div>
71     </section>
72 </body>
73 </html>
74
75
```

Fragmento formConsulta (en pruebas\fragmentos)

Este fragmento HTML es un excelente ejemplo de cómo usar Thymeleaf para crear componentes de interfaz de usuario reutilizables y dinámicos. Su propósito es definir la estructura de un formulario de consulta que se puede utilizar múltiples veces con diferentes configuraciones.

Este código define un fragmento de Thymeleaf llamado **formConsulta** que acepta tres parámetros: **colorForm**, **textoForm**, y **urlForm**. En lugar de tener que copiar y pegar el mismo código de formulario una y otra vez, este fragmento nos permite:

- a) **Evitar** la repetición de código (principio DRY - *Don't Repeat Yourself*).
- b) **Centralizar** la lógica visual del formulario en un solo lugar.
- c) **Hacer** el código más modular y fácil de mantener.

En resumen, este fragmento es un ejemplo de la modularidad en el desarrollo web con Spring Boot y Thymeleaf. Permite construir una interfaz de usuario flexible y escalable, donde un solo componente puede usarse para múltiples propósitos, minimizando el esfuerzo de desarrollo y facilitando futuras modificaciones.

```
73 <div th:fragment='formConsulta(colorForm, textoForm, urlForm)'>
74 <form method="post" th:action="@{${urlForm}}" class='was-validated'>
75 <div class='card p-2'>
76 <div class="card-header text-center text-white" th:classappend="${'bg-'+colorForm}">
77 <h5>[[${textoForm}]]</h5>
78 </div>
79 <div class="card-body">
80 <div class='mb-3'>
81 <label>[[#{consultas.precio.inferior}]]</label>
82 <input type="number" name='precioInf' th:value='${precioInf}'
83 class='form-control' min="0" required/>
84 </div>
85 <div class='mb-3'>
86 <label>[[#{consultas.precio.superior}]]</label>
87 <input type="number" name='precioSup' th:value='${precioSup}'
88 class='form-control' min="0" required/>
89 </div>
90 </div>
91 <div class="card-footer text-center">
92 <button type='submit' class='btn text-white' th:classappend="${'btn-'+colorForm}">
93 <i class='fa-solid fa-magnifying-glass'></i> [[${textoForm}]]
94 </button>
95 </div>
96 </div>
97 </form>
98 </div>
```


Ejemplo de las consultas

TechShop

Categorías

Productos

Consultas

Administración

Idioma

Su tienda en línea

Consultas Derivadas

Precio Inferior

Precio Superior

Consultas Derivadas

Consultas JPQL

Precio Inferior

Precio Superior

Consultas JPQL

Consultas SQL

Precio Inferior

Precio Superior

Consultas SQL

Acá va su Práctica #2



Monitor AOC 19

Lorem ipsum dolor sit amet consectetur...

€23 000,00

5 unidades



Monitor MAC

Quisque in ridiculus scelerisque plat...

€27 000,00

2 unidades



Monitor Flex 21

Natoque lacinia accumsan hendrerit ...

€24 000,00

5 unidades



Monitor Flex 36

Elementum sagittis dictumst leo cura...

€27 600,00

2 unidades

fidéлитas

Universidad

Nos vemos en semana 9

RETO: El reto de esta semana es hacer la tarea #2,
crea tú propia consulta
Puede ser: derivada, JPQL, SQL. Sólo 1 de estas
de lo que sea que guste.